Knowhow: Humanoid Diffusion Policy for Parcel Handling

Author: Chen Yanyu (陈彦宇) · **Period:** 25 May — 31 July 2026 · **Team:** MUJIN **Status at handoff:** working end-to-end dual-arm demo; not production-ready **Code:** `https://gitlab.mujin.com.cn/app/humanoid/-/tree/new-inference` · **Companion docs:** `Humanoid_Poc_Report`, `Humanoid_Poc_Deck`

---

0. How to read this

Twenty minutes: §3 (runtime model) → §7.2 (oscillation diagnosis)

First day: add §2 (bringing the system up), §4 (the runtime, module by module), §8 (operating parameters).

Before modifying the motion pipeline: read §7 in full.

---

1. What the system does

The robot picks a parcel off a table with two arms, lifts it, decides whether the shipping label is visible, flips the parcel ~180° if it is not, and places it label-up. The task is parcel orientation for downstream barcode scanning, chosen so the proof of concept mirrors an operation MUJIN performs in production.

The manipulation policy is a **diffusion policy** (Chi et al., Stanford REALab) trained by imitation from VR teleoperation demonstrations. It handles **grasp, lift, and flip**; placing, releasing, failure recovery, and start/stop gating are **scripted macros** triggered by YOLO perception.

The policy runs as a separate HTTP service (`POST /predict`) that consumes an observation dict and returns an action chunk; it never drives the robot directly. This repository holds everything around it: teleoperation, recording, dataset building, and the deployment stack that turns action chunks into safe motion. Training code and the policy server belong to the upstream diffusion-policy repo and live outside this repository. Hyperparameters such as prediction horizon, `n_obs_steps`, and the ResNet encoders are defined in the server's `task/dual_arm_ee_image.yaml`, of which this repo mirrors only the *shape contract*.

---

2. Platform and getting running

Item	Value	Where
Robot	AgiBot / 智元 精灵 G1 dual-arm humanoid	–
Vendor SDK	`a2d_sdk`, vendored in-repo; GDK v1.5.0 manual in `docs/`	`a2d_sdk/robot.py`
Cameras	head + 2 wrist (`head`, `hand_left`, `hand_right`)	`real_world/camera.py:31`
Policy	Diffusion policy, image-conditioned, dual-arm, EE-space	server-side
Kinematics	Pinocchio + in-repo URDF, replacing the SDK solver	`real_world/ik.py`
Sim	PyBullet (in-process)	`real_world/sim_backend.py`
Record rate	`RECORD_HZ = 10` Hz: policy action-row cadence	`real_world/timing.py:44`
Control rate	`CONTROL_HZ = 120` Hz: substep streaming to the arms	`real_world/timing.py:46`
Teleoperation	Pico VR headset over ZeroMQ	`pico_vr/`
Policy server	`10.12.11.144:9000` (scripts default to `:9001`)	`real_world/inference_controller.py:60`

> **Naming caution.** The vendor package is `a2d_sdk`, the vendor manual is titled "G01 GDK", the robot is the AgiBot Genie G1, and the URDF is `A2D.urdf`. A legacy `G1_SDK_ENV/` snapshot sits on disk, excluded from version control and no longer used. This naming has caused confusion before; in any external material, name the robot and the SDK separately.

Running it

```
# 1) Robot machine — full stack (needs a2d_sdk + ROS + the robot):
pip install -r requirements_gui.txt      # + `pip install ultralytics` for the YOLO gates
python robot_control_gui.py

# 2) Any laptop — hardware-free demo mode (synthetic robot, live webcam, no SDK/ROS):
python robot_control_gui.py --demo

# 3) SDK-free sim / IK tools:
python -m venv .venv && .venv/bin/pip install -r requirements_sim.txt
.venv/bin/python scripts/sim_infer_eval.py --source replay --recording recording001 \
    --recordings MDM_data_collection/recordings

# 4) Tests (also run automatically as a launch pre-flight — see below):
pytest tests/
```

Always run from the repo root so `real_world`, `gui`, `servers`, `a2d_sdk` resolve.

Launch pre-flight. `robot_control_gui.py:391` runs `tests/test_safety_invariants.py` *before the GUI builds*, and a regression blocks launch. `HUMANOID_SKIP_SAFETY_PREFLIGHT=1` bypasses it with an explicit warning; reserve it for exceptional cases.

The GUI (`robot_control_gui.py`, four tabs)

- **Console**: camera views and the inference panel. The intended sequence is printed in the UI (`gui/inference_panel.py:667`): ① *Start sim preview* → ② *Start auto-run*. For safety reasons, auto-run **refuses to start without the sim preview**.
- **Detector tuning**: live head-cam view with YOLO boxes, backed by the same `YoloGate` instance the robot uses, so a confidence change here applies to the running robot immediately.
- **VR teleop**: Pico teleoperation + demonstration recording.
- **Evaluation**: live success-rate dashboard reading `infer_logs/eval/*.jsonl`.

Manual step-through (① Infer once → ② Validate → ③ Release *n*) exists alongside auto-run and reaches the robot only when you press Release.

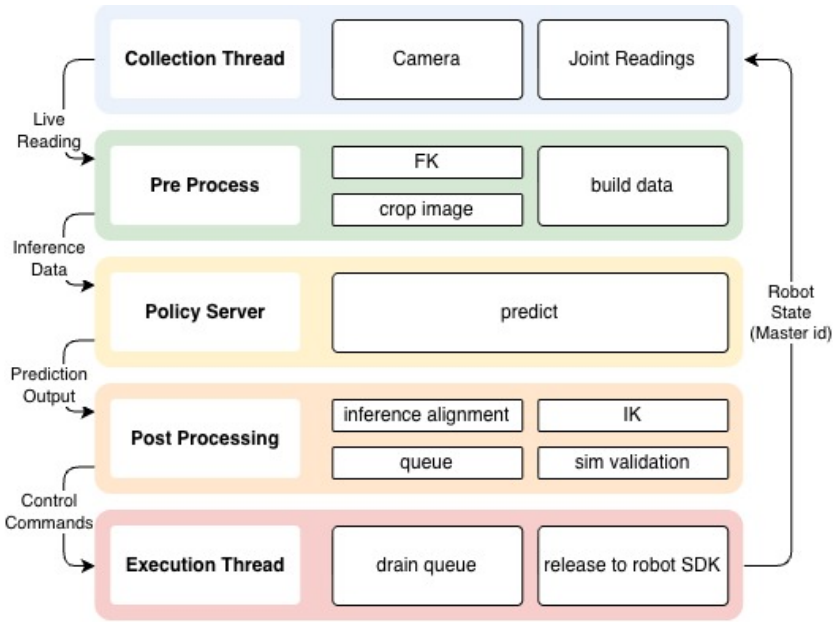
The SDK-free split

The SDK and ROS live **only on the robot machine**. IK, sim, camera hub, recorded-obs playback, timing, dataset building and the whole test suite import without them, via lazy imports (`real_world/__init__.py`, module `__getattr__`), a guarded SDK import (`real_world/humanoid_env.py:62`), and dependency injection (the sim runner injects a `_NoRobot` stand-in). This allows sim, evaluation and CI to run on a laptop, which roughly halved iteration time, as most debugging did not require the robot. Preserve this separation.

---

3. Mental model of the runtime

Runtime pipeline:



Observations and actions are mirror images. `real_world/build_data.py` constructs exactly the layout the policy trained on; `real_world/postprocess.py` is its inverse. Training (`MDM_data_collection/build_dataset.py:48`) *imports the same functions from* `build_data`, so train/deploy parity holds by construction. The one remaining gap is the image codec (mp4 at training, JPEG at inference). Changing one side requires changing the other and retraining. Most reported regressions in model quality traced back to divergence between the two.

Timing is owned by one module. `real_world/timing.py` is the single source of truth for `RECORD_HZ`, `CONTROL_HZ`, `SPEED_SCALE`, `MAX_JOINT_VEL`, `MAX_JOINT_STEP`, `RAMP_JOINT_STEP` and `WATCHDOG_MAX_JOINT_JUMP`. Its docstring records why: these constants once lived in three places (`humanoid_env`, `inference_controller`, `robot_data_collect`) and drifted, and execution timing was decoupled from recorded speed — each row gap was subdivided in *joint* space but drained in *time*, so wall-clock per row depended on how far the joints moved (big moves too slow, small moves too fast). That broke every merge whose "now" index assumes uniform substep spacing. **Introduce no second rate constant.**

The fix, encoded in the constants: each policy row expands to exactly `SUBSTEPS_PER_ROW = (CONTROL_HZ / RECORD_HZ) / SPEED_SCALE` substeps that are **uniform in time**. Inter-row wall-clock is always `ROW_DT`, independent of motion magnitude; smoothness is governed purely by `CONTROL_HZ`. `MAX_JOINT_STEP` serves as a **safety ceiling only**.

Alignment keys on absolute master row IDs. Every action row carries an absolute ID (the robot's own execution clock). Row *j* of a chunk anchored at *S* is at ID *S + j*. The release loop advances the clock as it pops (`postprocess.py:611`), `queue_status()` publishes `(current_row_id, queued_through)`, and every new inference merges against those. Absolute IDs survive variable inference latency, which clock-based synchronisation did not (§7.2).

Safety is layered and independent of the policy. The invariants are named C1–C7 / H1–H4 in the code and pinned by `tests/test_safety_invariants.py`:

ID	Guard	Where
C1	No sim running → nothing can ever reach the robot	`postprocess.py:642`
C2	Every released trajectory was stepped + self-collision-checked in sim	`postprocess.validate_chunk`
C3	E-stop is latched, actively holds pose, refuses release until reset	`humanoid_env.py:734`
C4	Bad/garbage joint reads fall back to last-good, never `None` mid-motion	`humanoid_env.py:859`
C5	Every dispatched substep ⩽ `MAX_JOINT_STEP` (both arms); larger becomes a ramp	`humanoid_env.py:1173`
C6	A single command jump > `WATCHDOG_MAX_JOINT_JUMP` (0.5 rad) **latches E-stop**	`humanoid_env.py:1138`
C7	A commanded EE outside the data-estimated safe box **latches E-stop**	`humanoid_env.py:1149`
H1	A validation can be released once, no re-release snap-back	`humanoid_env.py:226`

ID	Guard	Where
H2	Stale/frozen sensors abort the inference rather than predicting	`inference_controller.py:275`
H3	Orientation SLERP-smoothed row-to-row (`QUAT_ALPHA = 0.5`)	`postprocess.py:68`
H4	Per-arm workspace AABB rejects an out-of-box target before IK	`postprocess.py:70`
–	Firmware error/collision polled every release tick → E-stop	`humanoid_env.py:446`

C6 catches a large rotation; C7 catches a slow drift. C5 alone is insufficient because it bounds *velocity*: a bad IK branch-flip still executes, only slowly, producing a large unintended rotation. C6 bounds *displacement per command* and rejects it.

Camera streams are established on demand. The `CameraHub` (`real_world/camera.py`) holds one SDK camera object per camera and subscribes only while a consumer has called `request()`, evicting after `CAMERA_IDLE_TIMEOUT = 5.0 s`.

---

4. The runtime, module by module

Suggested reading order. Line counts indicate relative size.

Module	Lines	What it owns	Read it when
`real_world/timing.py`	102	Every rate and limit constant, with rationale	**Read first.**
`real_world/build_data.py`	152	Obs → `/predict` request; the shared pixel + EE-row transform	Changing anything the model sees
`real_world/postprocess.py`	786	The whole output pipeline + the robot queue + the merge buffer	Motion quality, smoothness, splicing
`real_world/ik.py`	354	Pinocchio DLS IK, per-arm models, rot6d↔quat, calibration loading	IK, frames, unreachable targets
`real_world/humanoid_env.py`	1225	SDK ownership, 3 threads, dispatch guards, gripper latch, recording	Hardware behaviour, E-stop, threads
`real_world/inference_controller.py`	874	The auto loop, server round-trip, macro call-ins, tracing	Loop order, when macros fire
`real_world/observer.py`	209	Obs buffers, freshness/staleness (H2), `get_obs`	Stale-obs aborts
`real_world/camera.py`	232	Dynamic camera subscription hub	Bandwidth, frames not arriving
`real_world/sim_backend.py`	683	PyBullet world + `validate()`	Validation failures
`real_world/sim_preview.py`	99	Sim-only preview loop (never touches the robot)	–
`real_world/recording.py`	206	Episode recorder (mp4 + npz + metadata)	Data collection
`real_world/recorded_obs.py`	127	Replay a recording as an obs source (SDK-free)	Offline eval

4.1 The data contract

Observation → server (`build_data.py:143`, matches `task/dual_arm_ee_image.yaml`):

Field	Shape	Content
`agentview_image`	(To,) b64 JPEG	head cam, **top-cropped** to 16:9 (keeps the workspace at the bottom)
`robot1_eye_in_hand_image`	(To,) b64 JPEG	left wrist, centre-cropped

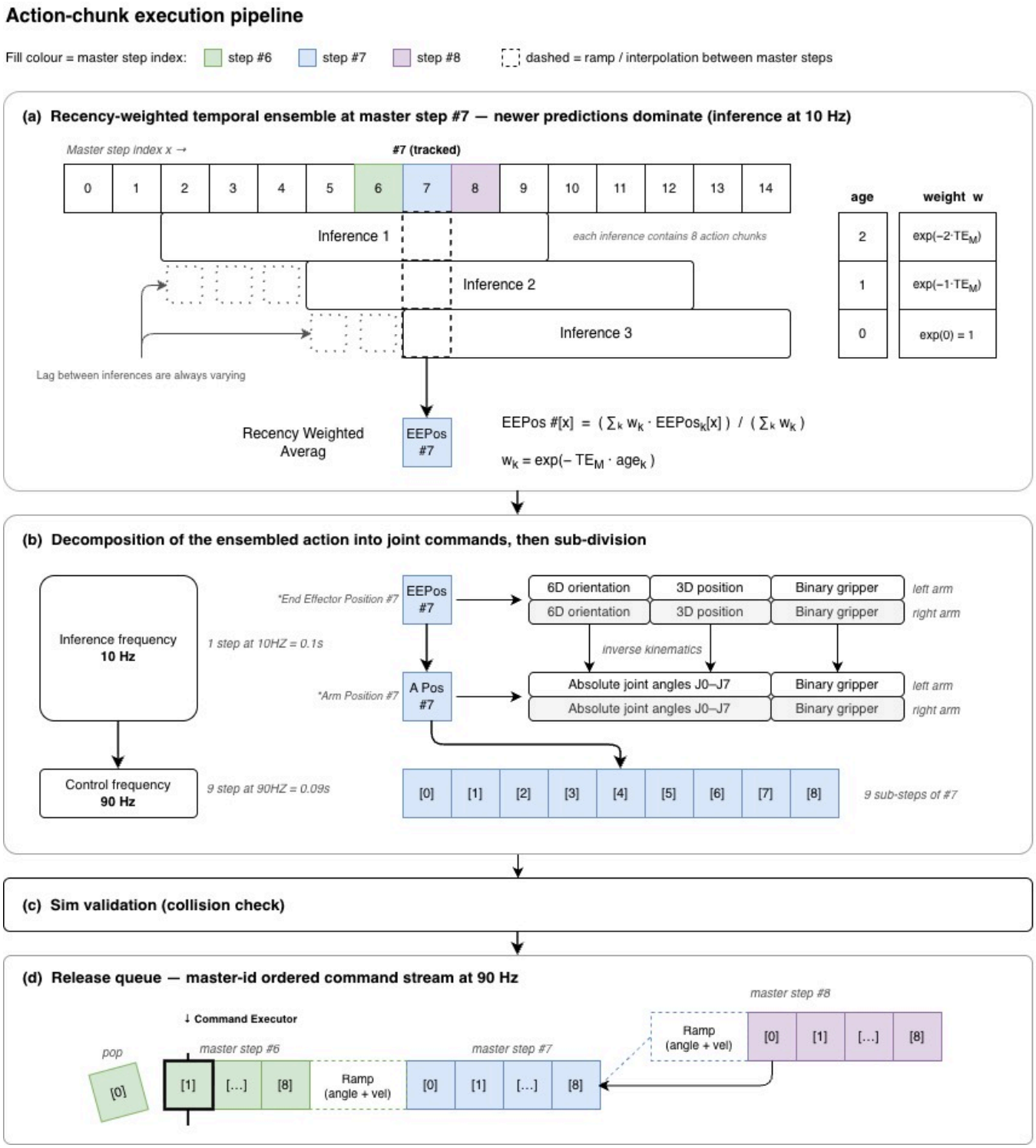
Field	Shape	Content
`robotr_eye_in_hand_image`	(To,) b64 JPEG	right wrist, centre-cropped
`robotl_eef_pos`	(To, 9)	left EE `[pos(3) + rot6d(6)]`
`robotr_eef_pos`	(To, 9)	right EE `[pos(3) + rot6d(6)]`
`robot0_grip`	(To, 2)	`[left, right]` gripper, **raw**, as recordings stored it

Action ← server: 20 columns, `L[pos3, rot6d6, grip1] ++ R[pos3, rot6d6, grip1]`. Gripper columns are **9 and 19**. (A 10-column left-only row still passes through for backward compatibility with the earlier single-arm policy.)

All images are cropped to 16:9 then resized to **`IMG_W × IMG_H = 256 × 144`**: every camera, both at training and at inference, through the same `preprocess_frame` (`build_data.py:71`). Cropping precedes JPEG encoding, so `imencode` runs on ~25× fewer pixels than the native 1280×800 head frame. Raising `AGENT_CROP_ZOOM` above 1.0 requires the **server to disable its own centre-crop**, otherwise the head frame is zoomed twice.

4.2 Output pipeline (`postprocess.py`)

1. **Gripper binarize** (`:230`): raw `[0, ~85]` → `{0, 1}` at `GRIPPER_CLOSE_THRESH = 10.0`, in place, the instant the chunk arrives, so gripper spikes never pollute the next inference's state context.
2. **Temporal-ensemble merge** (`:326`): two smoothing dimensions, both keyed on master ID: *(a)* across chunks, a recency-weighted mean of every buffered chunk's row at the **same absolute ID** (`w = exp(-TE_M · age)`), the ACT temporal ensemble; *(b)* along the ID axis, a symmetric Gaussian of half-width `TE_RADIUS`. IDs ⩽ `queued_through` are **frozen** (committed to the robot, read-only left-context so the seam stays continuous); IDs above are **mutable** and rebuilt every merge. Position and rot6d smooth linearly. **The gripper is never low-passed along the ID axis**, which would blur open/close timing; it carries the cross-chunk value re-thresholded at 0.5. Always average from *raw* chunks — re-buffering ensembled output compounds the smoothing.
3. **Dual-arm IK** (`:463`): per-row workspace gate (H4) → orientation SLERP (H3) → per-arm IK, warm-started row-to-row. `_ik_robust` (`:443`) solves from the live or chained seed; if that lands unreachable **or limit-pinned** (any joint within 0.10 rad of its limit, the signature of a contorted solution with the redundant DOF shoved into a stop), it retries from the **nominal training posture** (`config/nominal_arm_config.json`, the median of 67 106 recorded frames). This allows a *parked* right arm to resolve to a valid configuration rather than an extreme one. Sim evaluation never surfaced this, as it always seeds from warm recorded joints.
4. **Sim validation** (`:512`): substep through PyBullet, self-collision check, read back the sim-*achieved* joints. The sim returns an exact per-substep row index so master-ID tagging stays one-ID-per-row even when a velocity-capped row emits more than K substeps (`tests/test_tagging_exact.py` pins this).
5. **Queue splice** (`:631`): `append_actions` keeps `append_ahead_rows` policy rows queued ahead of the clock and appends **only IDs not yet queued**. Rows are only ever appended, never cleared; the release loop only pops the head. Steady state adds exactly one row per inference and needs no bridge (`seed_gap=True` makes the sim expand the queue-tail→row-0 gap so the trajectory is already continuous). A **velocity-merged bridge** fires only when validation took so long that the clock passed `start_id`; its cruise speed is the *mean* of the tail's exit velocity and the new rows' entry velocity, so the seam gains no spike.



An unreachable row in the streaming path **aborts the chunk and the arm holds** (`skip_unreachable= False` at `:674`). Dropping rows mid–stream makes the arm skip waypoints and jump the gap, which reads as jerky motion. The comment marks it a bisect candidate: a known lever if you revisit motion smoothness.

`auto_ingest_chunk` (`:727`) is the older queue–*replace* variant. `append_actions` is the live path.

4.3 Gripper handling

Three separate mechanisms are involved, and they are easily confused:

- **Obs side:** the model receives the **commanded** gripper value, non–binarised, so the policy sees true continuous state (`humanoid_env.py:244`, `_grip_obs_from_command = True`, closed mapped to the training raw scale `119.8`). Firmware read–back lags by *seconds*, which made the policy re–issue grasps it had already completed.
- **Action side:** binarised in post–processing (§4.2 step 1). Rich state in, clean decision out; this eliminated a large amount of command fluctuation.

- **Dispatch side:** an anti-chatter **change latch** (`humanoid_env.py:951`). When a channel's binary command flips, that state is committed and **locked for 20 master row IDs**; toggles inside the hold are ignored, so a policy that oscillates cannot re-grab. Cleared by `reset_grip_latch()` on E-stop, reset, auto start, and every retreat. **An uncleared latch will silently override an intended open with a latched-closed grasp.**

---

5. Data and training

5.1 Representation

	Representation	Why
Input	3D position + quaternion + gripper (+ joint angles, added later)	Quaternion is compact and efficient as an *input*. Joint angles were added because an EE-only model has poor awareness of arm configuration.
Output	3D position + **6D rotation** + gripper	6D rotation is continuous, which matters enormously for training stability. Quaternions double-cover and are discontinuous, making them poor regression targets.

End-effector space rather than joint space, on the diffusion-policy authors' own recommendation. Joint-angle output was trained and tested; EE output performed better. (A joint-space dataset builder survives at `MDM_data_collection/build_dual_arm_replay_buffer.py` if you want to revisit it.)

5.2 Recording → dataset pipeline

Recordings are captured at **30 Hz** (`MDM_data_collection/robot_data_collect.py:15`) and the dataset is built at **10 Hz** by keeping every 3rd frame. Each action row then spans 3× more motion, so the model makes more progress per inference; this was the fix for the stalling in §7.3.

The dataset stamps its own build rate into `meta/record_hz`, and `build_dataset.py:331` **refuses to build** if `--fps` disagrees with `timing.RECORD_HZ` unless you pass `--allow-hz-mismatch`. That invariant used to be a comment; keep it enforced. Obs rows stay **raw**; only the *action* target is Gaussian-smoothed (σ = 1.7 output-frames at 30 Hz, auto-scaled with `--fps`).

5.3 How much data, how many epochs

Dataset size	Behaviour
20 episodes	Roughly reproduces the motion; robustness collapses near the object. Any initial misalignment is unrecoverable.
50 episodes	Usable single-arm behaviour.
100 episodes	Reproduces fine detail from the demonstrations, including idiosyncrasies like releasing and using the lower gripper face to square the parcel before withdrawing.
200 episodes	Current dataset. Includes deliberately harder grasp conditions.

~50 demonstrations at 50—100 epochs already gives relative motion replication. **Diminishing returns set in above ~100 epochs**: the largest run was 200 epochs and showed no visible improvement. Training beyond that point requires a specific justification.

5.4 Tips for collecting high-quality data

This section has the greatest effect on model quality. The model is only as good as the consistency of the human operator, and none of it is captured in the code.

- **Exaggerate movements.** Subtle motions do not survive training. Movements that feel unnecessarily large during teleoperation reproduce correctly on the robot.

- **Be explicit about intent.** An end-to-end vision model infers no implicit intent. It learns only the patterns it observes repeated.
- **Keep approach margins consistent.** Concretely: if the head camera sees the parcel occupying x1—x2 and y1—y2 in frame, always approach with the gripper at x1 − n and x2 + n for a fixed n (~5 cm worked). This must hold for every demonstration, and matters most for parcels that sit low and close to the table, where the gripper has little clearance. **Inconsistency here was the single largest source of grasp failures.**

5.5 Image resolution

Current: everything cropped to 16:9 and resized to **256×144**, head cropped from the **top** (preserving the workspace at the bottom of the frame), wrists centred. Each camera gets its **own ResNet encoder** (server-side), on the theory that the head camera handles coarse positioning and the wrist cameras handle alignment and gripper decisions.

---

6. Design decisions and why

6.1 Hybrid policy + scripted macros

The policy does grasp, lift, flip. Everything else is a scripted macro running **inline in the auto-inference loop**, in this order (`inference_controller.py:803-858`):

```
E-stop check → package gate → no-flip place → flip place → recovery (inside _run_inference) → predict
```

Module	Role	Trigger
`package_gate.py`	Pause inference + park at home when there is no parcel	YOLO `package` class absent, **while idle only**
`no_flip_place.py`	Place as-is: warp a recorded joint path onto the live pose, move out, open, reverse back	YOLO `barcode` seen on **20 consecutive** scans, then grasped for 1.2 s
`flip_place.py`	Same, after the policy's ~180° flip	right wrist-roll rotated ⩾ 2.5 rad **since the grab**, held 0.8 s
`grasp_recovery.py`	Failed grasp: clear queue, open, retreat, let the policy re-plan	Wrist YOLO reports `closed-empty` while the right gripper is commanded closed
`retreat.py`	Torch-free, velocity-bounded retreat primitive shared by recovery and the unreachable handler	called by the above

The two place macros share the following design (`flip_place.py:119`):

- **Joint space throughout.** A fixed *end joint config* guarantees the same release point via FK with **no IK**, eliminating redundancy branch-flips and wrong-direction wrists.
- **Start adapts, end is fixed.** A decaying-offset warp re-anchors the recorded shape so `out[0] = live pose` and `out[-1] = the recorded release config`: `out[i] = rec[i] + (q_now − rec[0]) · (1 − i/(M−1))`.
- **The left arm is held** at its current pose throughout (`fwd[:, :7] = q_now[:7]`).
- **Everything is cleared before *and* after** (robot queue, staging, streaming cursor, merge buffer, grip latch), so nothing snaps when auto resumes. Forgetting any one of these was a recurring bug.
- The path files are built from real recordings by `scripts/build_release_path.py` (recording205 → no-flip, recording206 → flip), trimmed to the reach and lightly smoothed with endpoints pinned.

This split is deliberate. Placing is a geometrically constrained, highly repeatable motion for which a scripted path is more reliable and far cheaper than the demonstrations required to learn it, and it reserves the policy's capacity for the contact-rich portion of the task. State the cost explicitly: the system requires additional scripting to generalise to new placement locations.

6.2 Replacing the SDK's IK

The SDK ships a black-box IK solver (`set_end_effector_pose_control`). It was replaced with a transparent URDF + Pinocchio damped-least-squares solver (`real_world/ik.py`), one reduced 7-DOF model per arm with everything else locked, plus a constant `base_offset` SE3 mapping URDF `base_link` → the firmware reference frame.

Calibration is fitted offline by `scripts/fk_consistency_check.py` against recorded episodes: if a single constant X explains `SDK_FK(q) = X · our_FK(q)` across all frames, the firmware frame is effectively arm-relative and offline calibration is valid. The shipped fits:

	Position residual	Rotation residual
`config/fk_calibration.json` (left)	0.10 mm	0.008°
`config/fk_calibration_right.json` (right)	0.39 mm	0.116°

Independently validated by replaying recorded data through the solver: **over 400 target points, max deviation under 8 mm and 0.03 rad.**

The decisive advantage is that the same solver runs in the PyBullet check and on the robot. A black box permits no pre-execution trajectory validation, which makes every safety guarantee hollow. PyBullet was chosen over Isaac Lab for simplicity and because it runs on a GPU-less laptop.

Two solver details required significant debugging. The DLS step is **capped at 0.2 rad L2 per iteration** (`ik.py:252`): near a singular Jacobian the raw step can be several radians and diverge, with solutions landing >1 m / ~360° off target. And the solver returns the **best iterate seen**, because the iterate wobbles near the optimum and settles on the workspace boundary for unreachable targets. Strict IK tolerances (`reach_pos_tol = 0.02 m`) were restored after loose tolerances reintroduced jitter.

When a target is unreachable the arm **holds position**; skipping the waypoint produced unpredictable jumps.

6.3 Vision: why YOLO

Detecting the shipping label went through four generations, visible in the git history:

1. **Barcode library** (zxing-cpp): very unstable, heavy detection noise. The logic was sound (a waybill always carries a barcode), but the detector was unreliable. `BarcodeGate` still exists in `no_flip_place.py:359` as a swappable port.
2. **ArUco marker**: used only to validate the pipeline end to end, never a production solution.
3. **Printed text-block detection**: classical rule-based; *worse* than barcodes despite real tuning effort (max-detection-area filter, a noise floor, cropping the noisy top strip, and a whole live Detector Tuning GUI page built to tune it).
4. **YOLO**: roughly an order of magnitude more stable. Now used for everything.

Job	Class	Consumer	Weights
Parcel presence (gating)	`package`	`package_gate.py`	`real_world/assets/head_yolo.pt`
Label detection (flip vs no-flip)	`barcode`	`no_flip_place.YoloGate`	same model, shared
Gripper state (3-class)	`open` / `closed-gripped` / `closed-empty`	`grasp_recovery.py`	`real_world/assets/right_yolo.pt`

7. Debugging history

7.1 Stale end-effector data

Several issues caused stale robot state:

- Depth camera bandwidth saturation (~600 MB/s) blocked other DDS streams. This led to the on-demand `CameraHub`, which subscribes to cameras only when needed and releases them after inactivity.
- Unreliable SDK EE pose: joint states updated while the reported end-effector pose remained frozen. Resolved by computing EE pose via forward kinematics from live joint angles.

- Frozen joint states: joint-state topics streamed correctly only when a `Slam()` instance existed in the same process. The environment now instantiates `Slam()` solely to enable live joint updates.

7.2 Motion oscillation

Eliminating oscillation required multiple fixes:

- Corrected the SDK control method and reduced per-step motion to remove severe jitter.
- Stabilised gripper commands through binary state latching.
- Identified explicit trajectory merging (time-based or nearest-state) as a consistent source of oscillation. Visualising predicted trajectories showed the root cause: new trajectories lagged behind the executed motion.
- Replaced explicit merging with a buffered Gaussian smoothing pipeline, followed by an absolute master-ID timing system.
- Unified trajectory interpolation, fixed velocity scaling inconsistencies, and matched bridge velocities across trajectory boundaries.
- Removed expensive logging and redundant DDS polling from the 120 Hz control loop, restoring real-time performance.

The final stable configuration uses a 120 Hz control loop with Gaussian-smoothed trajectory buffering and parameters stored in `tuning_config.json`.

7.3 Inference rate

A 30 Hz policy produced insufficient motion between inference updates, causing the robot to stall. Retraining the policy at 10 Hz with end-effector targets resolved this, and 10 Hz is now the standard recording and inference rate.

7.4 Failure recovery

Recovery evolved from a simple home pose into a complete pipeline:

- Automatic homing before each run.
- YOLO-based grasp success detection.
- Automatic recovery after failed grasps.
- Merge buffer reset following recovery to resume inference correctly.
- Slower retreat speeds for smoother, safer motion.
- Five adaptive retreat waypoints selected according to the robot's progress through the grasp, reducing recovery time compared with always returning to a single home pose.

---

8. Operating and tuning reference

8.1 What each parameter controls

Parameter	Default / live	Effect	Where
`CONTROL_HZ`	120	**Primary motion-resolution parameter.** Higher → more substeps/row → finer motion. Must be an integer multiple of `RECORD_HZ`. Raise toward the SDK's sustainable waypoint rate (verify on hardware).	`timing.py:46`
`RECORD_HZ`	10	**Fixed by how the policy was trained**; changing it requires retraining.	`timing.py:44`
`speed_scale`	1.40 (JSON)	Fraction of demo speed. Lower → more substeps/row: smoother, slower, chunk lasts longer so the queue starves less. Alignment is ID-keyed, so any value is correct.	GUI "Speed (× demo)"

Parameter	Default / live	Effect	Where
`append_ahead_rows`	5 (JSON)	Rows queued ahead of the clock. **Constraint:** `n · SUBSTEPS_PER_ROW · STEP_TIME` must exceed one inference round-trip or the queue starves and the arm stalls. Larger = latency-robust but less reactive.	GUI "Look-ahead (rows)"
`te_radius`	6	Gaussian half-width along the ID axis; also the number of frozen rows kept as context.	GUI "Smoothness"
`te_sigma`	1.4	Gaussian σ; keep ⩽ `te_radius` or the kernel is clipped.	GUI "Blend strength"
`te_m`	0.123	Recency decay. Larger → trust the newest chunk more (reactive, rougher).	GUI "Recency"
`te_buffer_len`	8	How many recent raw chunks are averaged = max overlap depth.	GUI "Overlap depth"
`INFERENCE_HZ`	0	⩽ 0 → run back-to-back = maximum chunk overlap = most TE averaging. A positive cap cuts server load at the cost of smoothing.	`inference_controller.py:51`
`MAX_JOINT_VEL`	4.0 rad/s	**Safety cap only.** Real demo motion peaks ~5 rad/s (p99.9 ≈ 2.7); the rare ~17 rad/s spikes are sensor glitches.	`timing.py:67`
`WATCHDOG_MAX_JOINT_JUMP`	0.5 rad	C6 cutoff. Lower = stricter; 0 disables.	`timing.py:79`

8.2 Configuration and asset files

File	What	Regenerate with
`real_world/config/fk_calibration{,_right}.json`	Per-arm URDF→firmware `base_offset` SE3	`scripts/fk_consistency_check.py --side left|right`
`real_world/config/nominal_arm_config.json`	Median training posture; the IK fallback seed	median of `arm_joints` across recordings
`real_world/config/retreat_waypoints.json`	5 approach waypoints, both arms	`scripts/estimate_retreat_waypoints.py`
`real_world/assets/{flip,no_flip}_release_path.npy`	(M,14) scripted release paths	`scripts/build_release_path.py <recording> <out>`
`real_world/assets/head_yolo.pt`	`barcode` + `package` detector	annotate + train YOLO
`real_world/assets/right_yolo.pt`	3-class gripper-state detector	annotate + train YOLO
`real_world/assets/A2D_Omnipicker/A2D.urdf` + `meshes/`	Vendored URDF (self-contained)	vendor

File	What	Regenerate with
`postprocess.EE_SAFE_REGION_*`	C7 box, data-estimated + 0.12 m margin	`scripts/estimate_ee_region.py` → paste the tuples
`tuning_config.json`	Live operator tuning	the GUI, on every slider change

8.3 Diagnostics

All traces land in `infer_logs/` (`HUMANOID_TRACE_DIR` overrides).

Signal	Where	Read it for
`[infer] #N ... carried ids A..B | robot@id C (queued→D, lead L) | L-gap X cm`	console, per inference	Is auto commanding a jump away from the current pose? A large gap = anchoring away from the current state. `lead` = rows queued ahead.
`[release-timing] N substeps/s | total = recorder + firmware + dispatch`	console, ~1 Hz	If total ≫ `STEP_TIME` (8.3 ms), the release loop (not inference) is capping arm speed.
`[pipeline] append: ... (+N catch-up)`	console	A non-zero catch-up bridge means validation is falling behind the clock.
`SMOOTHNESS WARN: buffer |Δpos|max=...`	log	A per-row EE step > 3 cm near the seam, a bad merge. Diagnostic only, never clamps.
`buffer.jsonl`	always on	The smoothed run actually fed to the robot, per master ID. **The primary signal for smoothness analysis.**
`requests.jsonl`, `chunks.jsonl`	`HUMANOID_INFER_TRACE=1`	Raw proprioception sent / raw chunk returned.
`released_substeps.jsonl`, `live_joints.jsonl`	`HUMANOID_SUBSTEP_TRACE=1`	Commanded vs measured joints per master ID → tracking error. **Consumes real-time budget.**
`scripts/analyze_smoothness.py [DIR]`	offline	Collapses overlapping logs to one value per master ID; measures executed smoothness vs the single-chunk baseline (the policy's own smoothness = best achievable) and flags seam velocity spikes.

8.4 Offline evaluation without the robot

Script	What it proves
`scripts/sim_replay_eval.py serve` / `send`	Our IK driving the arm in PyBullet from recorded EE trajectories. No policy server.
`scripts/sim_infer_eval.py --source replay|policy`	The full deploy path (obs → server → IK → sim) against a recorded episode.
`scripts/sim_model_eval.py`	The trained policy driving both arms in sim on recorded perception, printing IK reachability, per-arm orientation error, max substep velocity vs cap, and predicted-vs-recorded EE deviation. **Use this to verify model behaviour before deployment.**
`scripts/fk_consistency_check.py`	Does our URDF FK match firmware FK? Produces the calibration.

Script	What it proves
`scripts/eval_trials.py --task flip_place`	Live trial scoring (SPACE = start, 1..N = outcome). Writes `infer_logs/eval/*.jsonl`, prints success rate with a Wilson 95 % CI and a failure-mode histogram. Never touches the robot.
`pytest tests/`	C1—C7 / H1 safety invariants, append/splice continuity, exact master-ID tagging, retreat waypoint selection.

Date modified: 31 July 2026